Trabalho Prático 1 - Indexação Geoespacial e Geometria Computacional Híbrida

Integrante(s):
Gabriel da Cunha Costa - 2024106026

Resumo

Este trabalho apresenta o desenvolvimento de uma aplicação web para geração, visualização e consulta de uma frota sintética de entregadores na cidade de Belo Horizonte. O sistema permite gerar entregadores distribuídos ao redor de estabelecimentos de alimentação, selecionar um restaurante e localizar os entregadores que estão dentro de um raio informado pelo usuário.

A busca geoespacial foi implementada utilizando Geohash em Base32 e uma Trie compacta, também chamada de Radix Trie. O Geohash transforma coordenadas geográficas em strings que preservam proximidade espacial, enquanto a Radix Trie permite recuperar eficientemente os entregadores associados a determinado prefixo geográfico. Para tratar o problema de pontos próximos que se encontram em células diferentes, a consulta também considera as oito células vizinhas à célula do restaurante.

Depois da recuperação dos candidatos pela Trie, a distância exata entre cada entregador e o restaurante é calculada por meio da fórmula de Haversine. Somente os entregadores que se encontram dentro do raio solicitado são apresentados como encontrados. Os demais permanecem visíveis no mapa em cinza.

Os dados dos estabelecimentos foram obtidos no Portal de Dados Abertos da Prefeitura de Belo Horizonte. Os entregadores foram gerados a partir de uma distribuição normal centrada nos restaurantes e, posteriormente, projetados para a via transitável mais próxima da malha do OpenStreetMap. A aplicação foi implementada em Python e disponibilizada por meio de uma interface web construída com Dash e Leaflet.

SUMÁRIO

1. Introdução 3

2. Base de dados e preparação dos restaurantes 3

3. Codificação Geohash 4

4. Tratamento das células vizinhas 5

5. Radix Trie 5

6. Geração dos entregadores 6

7. Filtragem geométrica e estimativa de chegada 7

8. Arquitetura e interface 7

9. Análise de complexidade 9

10. Avaliação experimental 10

    10.1 Metodologia 10

    10.2 Resultados 10

    10.3 Comparação visual entre raios 12

11. Testes 12

12. Limitações 13

13. Conclusão 13

Referências 14

1. Introdução

Aplicações de entrega precisam identificar rapidamente quais entregadores se encontram próximos de um estabelecimento. Uma solução baseada apenas na comparação direta entre o restaurante e todos os entregadores teria custo linear em relação ao tamanho da frota para cada consulta. Embora essa abordagem seja simples, sua eficiência diminui conforme a quantidade de entregadores cresce.

O objetivo deste trabalho foi desenvolver um mecanismo de indexação que reduzisse o conjunto inicial de candidatos antes do cálculo exato das distâncias. Para isso, as coordenadas dos entregadores foram codificadas como strings Geohash e armazenadas em uma Radix Trie desenvolvida especificamente para o projeto.

A ferramenta resultante permite:

- pesquisar restaurantes por nome, endereço ou bairro;
- gerar uma frota sintética com até 10.000 entregadores;
- ajustar o raio máximo de atendimento;
- localizar candidatos por meio de prefixos Geohash;
- filtrar os candidatos pela distância geográfica;
- visualizar restaurantes e entregadores em um mapa;
- consultar distância e estimativa de chegada de cada entregador;
- acompanhar métricas sobre a execução da busca.

As principais estruturas e algoritmos do trabalho, especialmente a codificação Geohash, a busca pelas células vizinhas e a Radix Trie, foram implementados manualmente. Bibliotecas externas foram empregadas apenas para a interface, transformação de coordenadas, manipulação de geometrias e obtenção da malha viária.

---

2. Base de dados e preparação dos restaurantes

A base utilizada foi o conjunto “Atividade Econômica”, disponibilizado pela Prefeitura de Belo Horizonte em seu Portal de Dados Abertos. O arquivo contém informações sobre atividades econômicas registradas no município, incluindo nome do estabelecimento, endereço, bairro, código CNAE e localização geográfica.

O arquivo bruto utilizado possuía 572.224 registros. Para selecionar estabelecimentos relacionados a restaurantes e serviços de alimentação, foram mantidos os registros cujo código CNAE começa com 56112.

O processamento foi realizado linha a linha para evitar a necessidade de manter o arquivo bruto inteiro na memória. As principais etapas foram:

1. identificação automática do separador do arquivo CSV;
2. normalização do código CNAE;
3. filtragem dos registros da classe 56112;
4. leitura da geometria no formato WKT;
5. conversão das coordenadas de EPSG:31983 para EPSG:4326;
6. validação dos campos obrigatórios;
7. gravação de um novo arquivo CSV simplificado.

O arquivo processado contém os seguintes campos:

- identificador;
- nome do estabelecimento;
- endereço;
- bairro;
- latitude;
- longitude.

Ao final do processamento, foram obtidos 30.118 estabelecimentos. O procedimento descartou 542.106 registros que não pertenciam à classe selecionada e não encontrou geometrias inválidas ou registros duplicados entre os restaurantes mantidos.

O sistema carrega o arquivo processado em um repositório em memória. Para facilitar a pesquisa na interface, nome, endereço e bairro são normalizados, desconsiderando diferenças entre letras maiúsculas, minúsculas e acentos.

---

3. Codificação Geohash

Geohash é uma técnica que transforma um par de coordenadas geográficas em uma string. Pontos geograficamente próximos tendem a compartilhar um prefixo maior, permitindo utilizar operações sobre strings como aproximação para consultas espaciais.

A implementação divide alternadamente os intervalos de longitude e latitude. Inicialmente, a longitude pertence ao intervalo de −180° a 180° e a latitude ao intervalo de −90° a 90°. A cada divisão, é registrado um bit indicando se a coordenada está na metade inferior ou superior do intervalo atual.

Os bits da longitude e da latitude são intercalados. A cada grupo de cinco bits, o valor resultante é convertido para um caractere do alfabeto Base32 do Geohash:

0123456789bcdefghjkmnpqrstuvwxyz

O projeto limita os Geohashes a oito caracteres. Quanto maior o prefixo, menor e mais precisa é a região representada.

Além da codificação, foi implementada a operação inversa necessária para determinar os limites geográficos de uma célula. A partir desses limites, o sistema calcula a altura e a largura aproximadas da região representada pelo prefixo.

A precisão utilizada em cada consulta é escolhida dinamicamente de acordo com o raio informado. O sistema seleciona o prefixo mais específico cuja célula ainda possui altura e largura suficientes para cobrir o raio solicitado. Assim, raios menores utilizam prefixos mais longos, enquanto raios maiores utilizam prefixos mais curtos.

---

4. Tratamento das células vizinhas

Um ponto importante do Geohash é que dois locais fisicamente próximos podem possuir prefixos diferentes quando estão separados pela borda de uma célula. Pesquisar somente o prefixo do restaurante poderia, portanto, deixar de encontrar entregadores próximos.

Para evitar esse problema, a aplicação consulta nove células:

- a célula que contém o restaurante;
- a célula ao norte;
- a célula ao sul;
- a célula a leste;
- a célula a oeste;
- as quatro células diagonais.

Os centros das células vizinhas são obtidos a partir das dimensões da caixa geográfica decodificada. Esses centros são novamente codificados com o mesmo tamanho de prefixo usado para o restaurante.

Os IDs recuperados nas nove consultas são inseridos em um dicionário, o que remove automaticamente possíveis repetições. O resultado dessa etapa é o conjunto de candidatos que será submetido à filtragem geométrica exata.

---

5. Radix Trie

Os Geohashes dos entregadores são armazenados em uma Trie compacta, também conhecida como Radix Trie. Diferentemente de uma Trie convencional, em que cada aresta normalmente armazena apenas um caractere, a Radix Trie agrupa sequências de caracteres que não possuem ramificações.

Cada nó da estrutura contém:

- o rótulo associado à aresta;

- um dicionário de filhos;
- um conjunto ordenado de IDs dos entregadores pertencentes à subárvore.

Os filhos são indexados pelo primeiro caractere de seus rótulos. Durante uma inserção, o algoritmo calcula o maior prefixo comum entre o trecho restante da chave e o rótulo da aresta encontrada.

Quando a correspondência é parcial, a aresta é dividida e um novo nó intermediário é criado. Dessa maneira, a estrutura mantém apenas as ramificações necessárias.

Os IDs também são armazenados nos nós ancestrais percorridos durante a inserção. Essa decisão aumenta o consumo de memória, mas permite que a busca por prefixo devolva diretamente todos os entregadores da região, sem precisar percorrer toda a subárvore.

A busca possui custo estrutural proporcional ao tamanho do prefixo. Considerando a construção da coleção de resultados, sua complexidade é:

$O(m + k)$

em que m representa o tamanho do prefixo pesquisado e k a quantidade de entregadores retornados.

Como o Geohash possui no máximo oito caracteres neste projeto, o tamanho das chaves é limitado. A compactação continua sendo importante por diminuir a quantidade de nós e demonstrar a aplicação de estruturas de dados para strings em um problema geográfico.

---

6. Geração dos entregadores

A frota utilizada pela aplicação é sintética. O usuário informa a quantidade de entregadores e pode definir uma semente para permitir a reprodução dos resultados.

A quantidade solicitada é distribuída de maneira equilibrada entre os restaurantes disponíveis. Para cada entregador, uma posição inicial é amostrada por meio de uma distribuição normal bidimensional centrada nas coordenadas do restaurante correspondente.

Antes da amostragem, as coordenadas são transformadas para o sistema EPSG:31983, no qual as unidades são expressas em metros. Isso permite utilizar um desvio-padrão de 800 metros diretamente nos eixos horizontal e vertical.

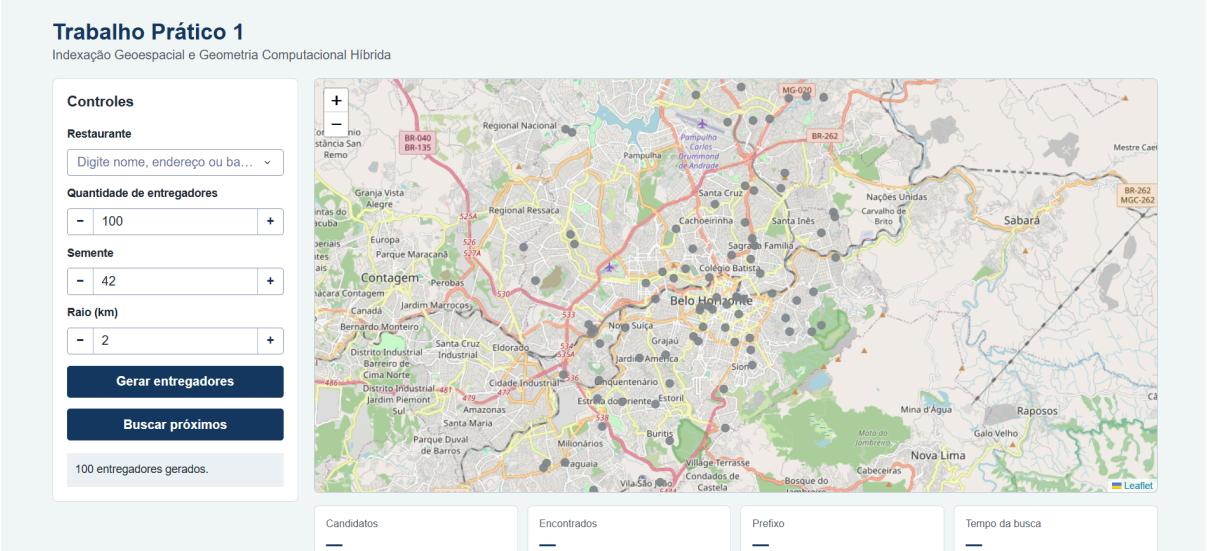
Uma geração puramente matemática poderia posicionar entregadores dentro de construções, quarteirões, parques ou corpos d’água. Por isso, cada posição gerada é corrigida utilizando a malha viária dirigível de Belo Horizonte obtida do OpenStreetMap.

Para cada ponto, o sistema:

1. localiza a aresta transitável mais próxima;
2. projeta o ponto sobre a geometria dessa aresta;
3. converte a posição corrigida novamente para latitude e longitude;
4. calcula o Geohash com oito caracteres;
5. insere o entregador na Radix Trie.

A malha viária é salva em um arquivo de cache. Assim, o download pelo OSMnx é necessário somente na primeira execução. Nas execuções posteriores, a aplicação reutiliza o grafo armazenado localmente.

Ao gerar uma nova frota, o sistema cria primeiro uma nova coleção e uma nova Trie. O estado anterior somente é substituído depois que toda a geração termina corretamente. Isso evita deixar a aplicação com uma frota parcialmente construída caso ocorra algum erro.



---

7. Filtragem geométrica e estimativa de chegada

A recuperação por prefixo produz apenas um conjunto aproximado de candidatos. Para verificar quais entregadores realmente estão dentro do raio, é utilizada a distância de Haversine.

Considerando duas coordenadas geográficas, a fórmula empregada é:

$$a = \text{sen}^2(\Delta\varphi/2) + \cos(\varphi_1) \times \cos(\varphi_2) \times \text{sen}^2(\Delta\lambda/2)$$

$$d = 2R \times \text{arcsen}(\sqrt{a})$$

Nessa expressão, φ representa a latitude em radianos, λ representa a longitude e R é o raio médio da Terra, definido como 6.371,0088 km.

Um entregador é considerado encontrado quando sua distância é menor ou igual ao raio selecionado. Os resultados são ordenados primeiramente pela distância e, em caso de empate, pelo identificador.

A estimativa de tempo de chegada, ou ETA, considera uma velocidade média constante de 20 km/h:

ETA = (distância / velocidade) × 60

O ETA é uma aproximação. Ele não considera trânsito, semáforos, sentido das vias ou o percurso real entre os pontos.

---

8. Arquitetura e interface

O projeto foi dividido em dois módulos principais. O primeiro contém a lógica de domínio, incluindo Geohash, Radix Trie, preparação dos restaurantes, geração dos entregadores, cálculos geográficos e serviço de consulta. O segundo contém a aplicação web e os componentes visuais.

A organização principal é:

```
src/tp1/
├── models.py
├── geohash.py
├── trie.py
├── geometry.py
├── restaurants.py
├── simulation.py
└── service.py

dashboard/
├── app.py
├── components.py
├── presentation.py
└── assets/
    └── style.css
```

A interface foi desenvolvida com Dash. O mapa utiliza Leaflet e dados cartográficos do OpenStreetMap, enquanto a tabela utiliza Dash AG Grid.

O usuário pode controlar:

- o restaurante da consulta;
- a quantidade de entregadores;
- a semente da geração;
- o raio de atendimento.

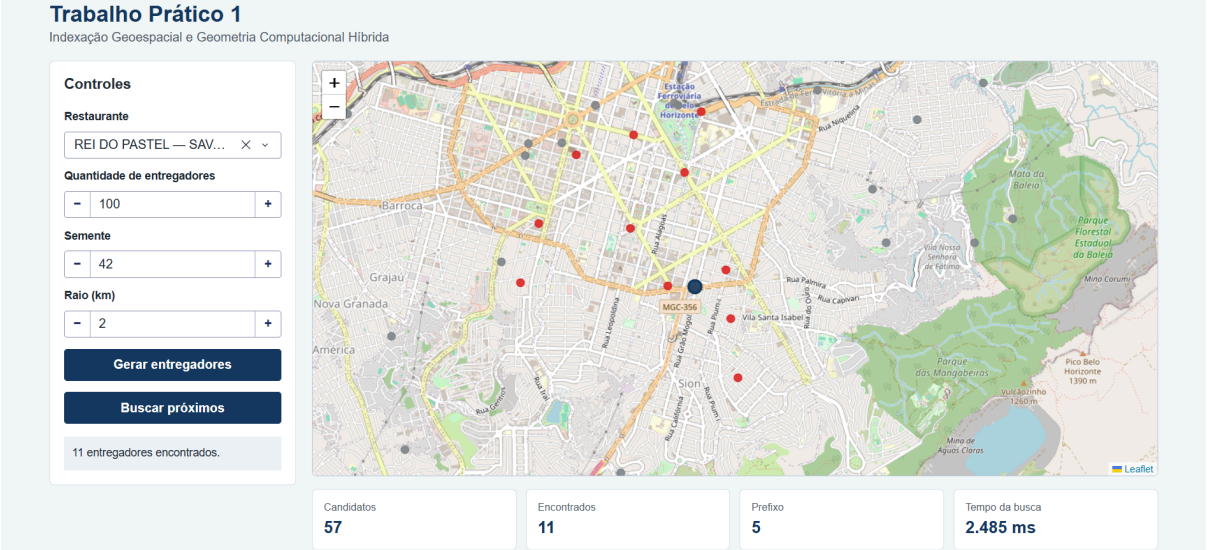
O restaurante selecionado é representado em azul-escuro. Os entregadores encontrados aparecem em vermelho e os demais aparecem em cinza.

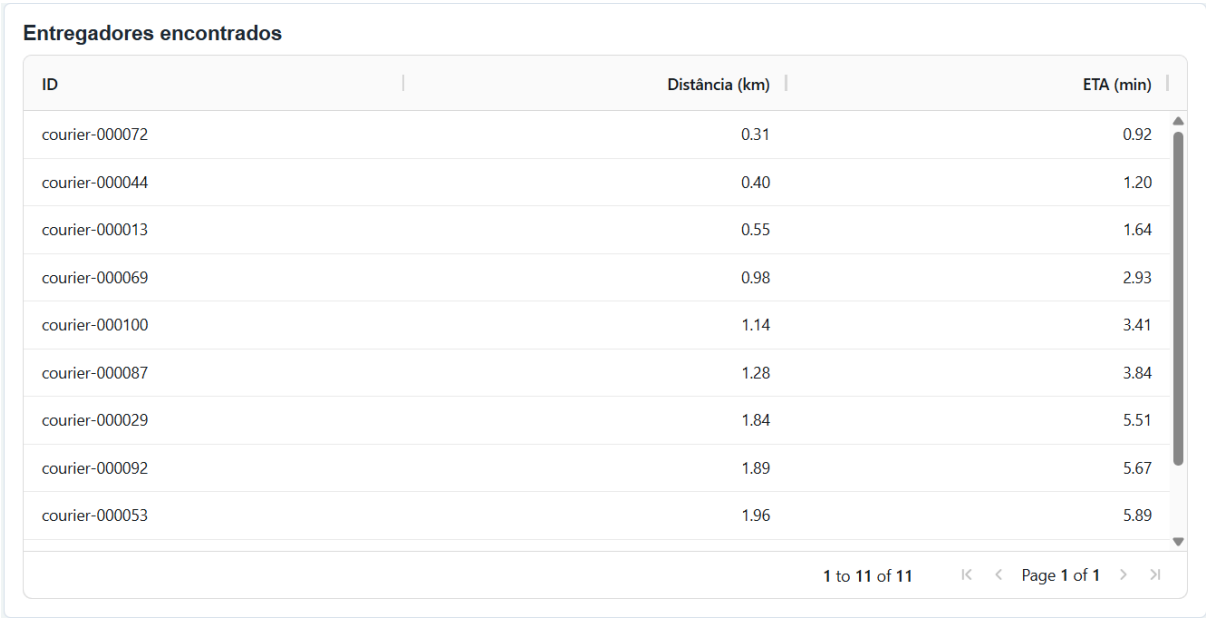
Após uma consulta, a interface apresenta quatro indicadores:

- **Candidatos:** quantidade de IDs recuperados da Radix Trie nas nove células consultadas;
- **Encontrados:** quantidade de candidatos cuja distância exata está dentro do raio;
- **Prefixo:** número de caracteres do Geohash usado na consulta;
- **Tempo da busca:** tempo gasto pelo backend para consultar os prefixos, filtrar os candidatos, ordenar os resultados e preparar os IDs descartados.

O tempo exibido não inclui a geração da frota, o carregamento da malha viária nem o tempo de renderização do navegador.

A tabela exibe o identificador, a distância em quilômetros e o ETA em minutos para cada entregador encontrado.





Entregadores encontrados

ID	Distância (km)	ETA (min)
courier-000072	0.31	0.92
courier-000044	0.40	1.20
courier-000013	0.55	1.64
courier-000069	0.98	2.93
courier-000100	1.14	3.41
courier-000087	1.28	3.84
courier-000029	1.84	5.51
courier-000092	1.89	5.67
courier-000053	1.96	5.89

1 to 11 of 11 Page 1 of 1

---

9. Análise de complexidade

Seja:

- N a quantidade total de entregadores;
- m o tamanho do prefixo Geohash;
- C a quantidade de candidatos recuperados;
- R a quantidade de entregadores dentro do raio.

As principais operações possuem as seguintes complexidades:

Operação	Complexidade
Codificação de uma coordenada	O(m)
Inserção na Radix Trie	O(m)
Busca por um prefixo	O(m + k)
Consulta das nove células	O(m + C)
Filtragem pela distância	O(C)
Ordenação dos encontrados	O(R log R)
Geração da lista de descartados	O(N)

O fator nove das células pesquisadas é constante e, portanto, não altera a ordem assintótica.

A etapa central de localização e filtragem pode ser representada por:

$O(m + C + R \log R)$

Entretanto, a implementação também percorre a frota completa para informar à interface quais entregadores devem aparecer em cinza. Assim, considerando toda a resposta preparada para o mapa, o custo é:

$O(m + C + R \log R + N)$

Esse percurso completo é uma consequência da escolha de manter todos os entregadores visíveis no mapa. Em uma aplicação que exibisse somente os entregadores encontrados, esse termo adicional poderia ser removido.

Em relação à memória, os IDs são agregados nos nós ancestrais da Radix Trie. No pior caso, o consumo associado aos IDs pode chegar a $O(N \times m)$. Como neste projeto m está limitado a oito caracteres, esse custo permanece linear em relação ao tamanho da frota, com um fator constante limitado.

---

10. Avaliação experimental

10.1 Metodologia

Os experimentos foram executados utilizando o script benchmark_queries.py. Foram geradas frotas com 100, 1.000 e 10.000 entregadores.

Em todas as execuções foram usados:

- semente igual a 42;
- desvio-padrão igual a 800 metros;
- restaurante de ID 1270, denominado “LANCHONETE FLOR DO MAR LTDA”;
- raios de 0,5 km, 1 km, 2 km e 5 km;
- nove células Geohash por consulta.

Os tempos representam somente a operação de busca executada pelo serviço. A geração da frota e o carregamento do grafo viário não foram incluídos.

Cada combinação foi medida uma vez. Consequentemente, os valores representam execuções observadas no ambiente local e não uma média estatística. Pequenas variações são esperadas em outras máquinas ou em novas execuções.

Ambiente de execução:
[PROCESSADOR]
[QUANTIDADE DE MEMÓRIA RAM]
[SISTEMA OPERACIONAL]
[VERSÃO DO PYTHON]

10.2 Resultados

Frota	Raio (km)	Prefixo	Candidatos	Encontrados	Tempo (ms)
100	0,5	6	2	0	0,4494
100	1,0	5	32	1	0,2517
100	2,0	5	32	3	0,2150
100	5,0	4	100	16	0,2654
1.000	0,5	6	17	1	0,5051
1.000	1,0	5	387	6	0,6146
1.000	2,0	5	387	35	0,6044
1.000	5,0	4	1.000	188	1,3779
10.000	0,5	6	220	24	2,1705
10.000	1,0	5	3.822	106	7,3679
10.000	2,0	5	3.822	409	9,6615
10.000	5,0	4	10.000	1.944	17,1838

Os resultados mostram que a precisão do prefixo diminui quando o raio aumenta. Para 0,5 km foi utilizado um prefixo com seis caracteres, enquanto o raio de 5 km utilizou somente quatro caracteres.

Prefixos menores representam células maiores e recuperam mais candidatos. Nas consultas de 5 km, todos os entregadores da frota foram recuperados como candidatos. Ainda assim, apenas aqueles cuja distância calculada pela fórmula de Haversine estava dentro do raio foram considerados encontrados.

As consultas com raios de 1 km e 2 km utilizaram o mesmo tamanho de prefixo e, por isso, recuperaram os mesmos candidatos. A quantidade final de encontrados foi diferente devido à filtragem geométrica.

Na maior configuração, com 10.000 entregadores, os tempos observados variaram entre aproximadamente 2,17 ms e 17,18 ms. O maior tempo ocorreu na consulta de 5 km, que recuperou toda a frota como candidata e encontrou 1.944 entregadores.

Os números demonstram que o uso da Trie e dos prefixos Geohash reduz significativamente a quantidade de cálculos de distância nas consultas com regiões menores.

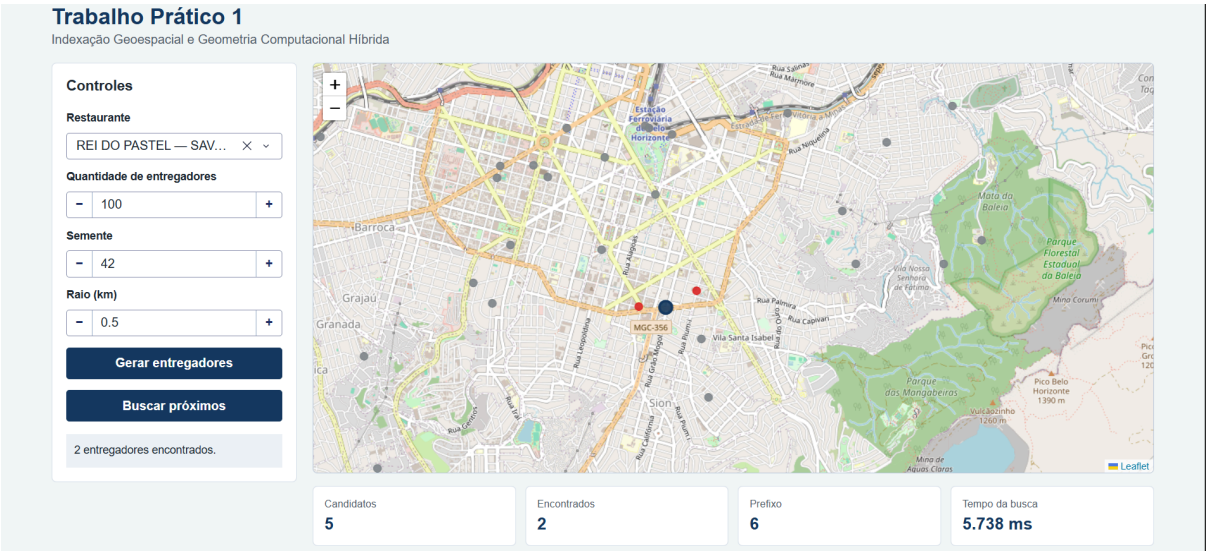
[INSERIR GRÁFICO 1 AQUI — RECOMENDADO]

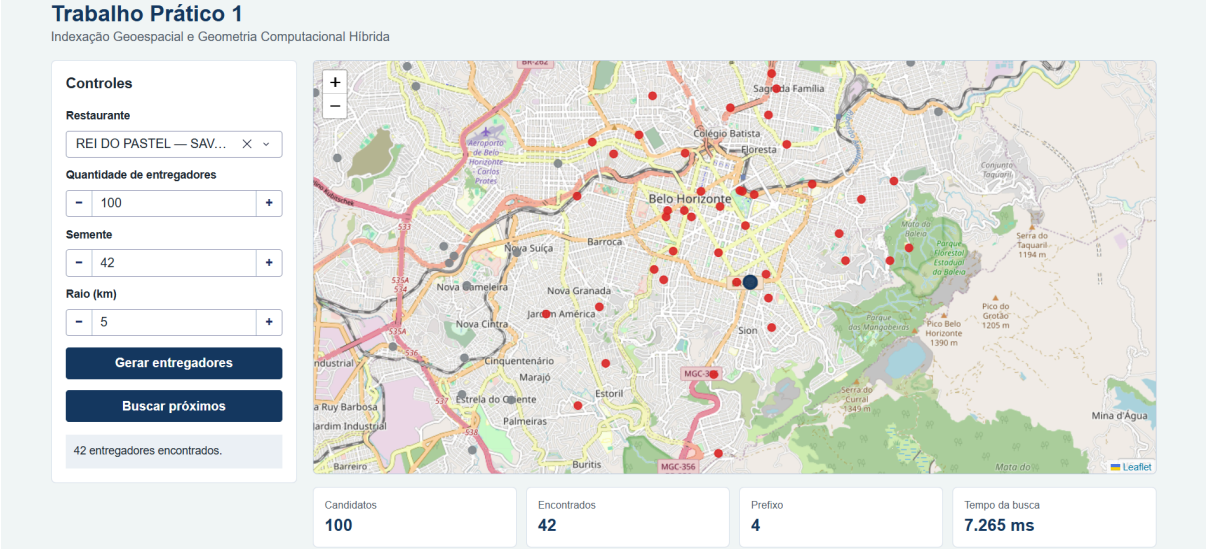
Crie um gráfico de linhas com:

- eixo horizontal: tamanho da frota;
- eixo vertical: tempo em milissegundos;
- uma linha para cada raio.

Gráfico 1 — Tempo de busca em função do tamanho da frota e do raio consultado. Fonte: elaboração própria.

10.3 Comparação visual entre raios





---

11. Testes

Foram criados testes automatizados para os principais módulos da aplicação. A suíte contém 104 testes e pode ser executada com o comando:

python -m pytest -q

Os testes cobrem:

- validação dos modelos;
- codificação e decodificação Geohash;
- cálculo das células vizinhas;
- escolha da precisão conforme o raio;
- inserção e busca na Radix Trie;
- divisão de arestas da Trie;
- cálculo da distância de Haversine;
- cálculo do ETA;
- preparação da base de restaurantes;
- pesquisa por nome, endereço e bairro;
- geração determinística por semente;
- distribuição dos entregadores;
- correção das posições para as vias;
- geração e substituição da frota;
- busca de candidatos;
- filtragem por distância;
- validação dos parâmetros;
- componentes e callbacks da interface;
- scripts de preparação, geração e benchmark.

Na última execução, todos os testes foram aprovados:

104 passed

Os testes automatizados não dependem de acesso à internet. Para validar a projeção dos entregadores nas vias, é utilizado um grafo sintético durante os testes.

---

12. Limitações

A principal limitação é que o ETA é calculado a partir da distância em linha reta e de uma velocidade constante. Uma estimativa mais realista exigiria calcular rotas sobre a malha viária e considerar trânsito, sentido das vias e variações de velocidade.

A distribuição normal é uma aproximação simplificada do comportamento de uma frota. Em uma aplicação real, a localização dos entregadores seria atualizada por dispositivos móveis, e a demanda poderia variar conforme horário, região e histórico de pedidos.

O estado atual da frota é mantido na memória do processo. A aplicação foi projetada para demonstração local e utilização por uma pessoa. Uma versão multiusuário precisaria armazenar sessões ou frotas separadas em um banco de dados.

Embora a consulta por prefixos reduza o conjunto usado na filtragem geométrica, a interface exibe todos os entregadores descartados em cinza. Por isso, o serviço ainda percorre a frota para construir essa representação.

Também existe dependência da qualidade e da atualização dos dados fornecidos pela Prefeitura de Belo Horizonte e pelo OpenStreetMap.

---

13. Conclusão

O trabalho resultou em uma aplicação funcional para geração e consulta de uma frota sintética de entregadores em Belo Horizonte. A solução combina uma estrutura de dados voltada ao tratamento de strings com técnicas de geometria computacional.

A codificação Geohash permitiu representar regiões geográficas por prefixos. A Radix Trie possibilitou recuperar os entregadores associados a essas regiões em tempo proporcional ao tamanho do prefixo e ao número de resultados. A consulta das células vizinhas evitou a perda de entregadores próximos às bordas das células.

A fórmula de Haversine realizou a filtragem exata dos candidatos, enquanto a utilização da malha viária do OpenStreetMap garantiu que as posições sintéticas fossem projetadas sobre vias transitáveis.

Nos experimentos realizados, as consultas permaneceram abaixo de 18 ms mesmo com uma frota de 10.000 entregadores. Os resultados também evidenciaram a relação entre o tamanho do raio, o comprimento do prefixo e a quantidade de candidatos recuperados.

Por fim, a interface integra as etapas de geração, busca e visualização, permitindo observar tanto o funcionamento do algoritmo quanto seus resultados de forma interativa.

Referências

BELO HORIZONTE. Prefeitura Municipal. **Atividade Econômica**. Portal de Dados Abertos da Prefeitura de Belo Horizonte. Disponível em: https://dados.pbh.gov.br/pt_BR/dataset/atividade-economica. Acesso em: 19 out. 2026.

BOEING, Geoff. **OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks**. Computers, Environment and Urban Systems, v. 65, p. 126–139, 2017.

OPENSTREETMAP CONTRIBUTORS. **OpenStreetMap**. Disponível em: https://www.openstreetmap.org/. Acesso em: 19 out. 2026.

PLOTLY. **Dash documentation**. Disponível em: https://dash.plotly.com/. Acesso em: 19 out. 2026.

PYTHON SOFTWARE FOUNDATION. **Python documentation**. Disponível em: https://docs.python.org/3/. Acesso em: 19 out. 2026.